

Informe del módulo RAG

CSDI RAG Engine

1 Módulo RAG

El módulo RAG (*Retrieval-Augmented Generation*) es el componente central del sistema. Su responsabilidad es tomar la pregunta en texto libre de un usuario y producir una respuesta en lenguaje natural fundamentada en evidencia extraída del corpus o de la web. Para lograrlo, coordina todos los demás módulos del sistema: recuperación híbrida, detección de insuficiencia, búsqueda web, reordenamiento y generación con un modelo de lenguaje.

El módulo no implementa ninguna de esas capacidades directamente: las delega a componentes especializados y orquesta su ejecución en el orden correcto. Su función es garantizar que el modelo de lenguaje reciba siempre el conjunto de fragmentos más relevante disponible, ya sea del corpus local, del caché web o de una búsqueda externa reciente. Esta separación entre orquestación y capacidades específicas permite que cada componente evolucione de forma independiente sin afectar al módulo.

La motivación de usar RAG en lugar de un modelo de lenguaje sin contexto es concreta: los modelos de lenguaje tienen un conocimiento estático limitado a su fecha de entrenamiento, no pueden acceder al corpus privado del sistema y tienden a generar afirmaciones plausibles pero incorrectas cuando no tienen evidencia. Al forzar al modelo a construir su respuesta exclusivamente sobre fragmentos recuperados, el sistema ancla las respuestas a fuentes verificables y permite atribuir cada afirmación a un documento específico.

Diseño general del pipeline. El pipeline RAG se organiza en seis etapas secuenciales, cada una condicionada al resultado de la anterior:

1. **Recuperación inicial:** el `HybridRetriever` busca en el corpus los fragmentos más relevantes para la consulta.
2. **Evaluación de suficiencia:** el `InsufficiencyDetector` decide si la evidencia local es suficiente para responder.
3. **Consulta al caché web:** si la evidencia local es insuficiente, se busca en fragmentos de búsquedas web anteriores.
4. **Búsqueda web externa:** si el caché tampoco es suficiente, el `WebSearchOrchestrator` ejecuta una búsqueda externa, descarga documentos y los indexa en el caché.
5. **Selección y reordenamiento:** el `CrossEncoderReranker` reordena los fragmentos candidatos y selecciona los mejores para el prompt.
6. **Generación:** el `LLMClient` recibe el prompt construido con los fragmentos seleccionados y genera la respuesta.

Las etapas 2 a 4 son condicionales: si el corpus local es suficiente, las etapas 3 y 4 no se ejecutan. Si el componente de búsqueda web no está configurado, la etapa 4 no se ejecuta y el pipeline responde con la mejor evidencia disponible. Esta estructura garantiza que el caso común (corpus suficiente) sea el más rápido, y que el costo adicional de la búsqueda web solo se incurra cuando aporta valor.

2 Recuperación inicial

La recuperación inicial invoca al `HybridRetriever` con la pregunta del usuario y solicita un conjunto de candidatos de tamaño $k_{\text{candidate}}$. Cuando el reordenador está habilitado, este valor es `reranker_candidate_k=30`; sin reordenador, se usa directamente `context_chunks=15`. Esta distinción es intencional: recuperar más candidatos que los que se enviarán al prompt tiene sentido solo si hay una etapa de reordenamiento posterior que filtre el conjunto.

Si HyDE está habilitado (`HYDE_ENABLED=true`), la consulta es expandida antes de la búsqueda vectorial: el `LLMClient` genera un párrafo hipotético que respondería la pregunta, y ese párrafo se pasa al recuperador vectorial en lugar del texto original de la consulta. El recuperador léxico BM25 continúa operando sobre la consulta original. HyDE está desactivado por defecto porque viola el principio de diseño del sistema de reservar el modelo generativo exclusivamente para la etapa de generación de respuesta; su presencia como opción configurable permite evaluar su impacto sobre la calidad de recuperación en el corpus real.

Los fragmentos recuperados se devuelven junto con sus hits de fusión, que incluyen el campo `found_by`: un conjunto que indica si el fragmento fue encontrado por BM25, por el recuperador vectorial, o por ambos. Este campo se usa más adelante para determinar el *método de recuperación* de cada fuente citada en la respuesta.

3 Evaluación de suficiencia e integración de búsqueda web

Tras la recuperación inicial, el pipeline entrega los fragmentos al `InsufficiencyDetector`, que calcula una puntuación de confianza local basada en cinco señales: cobertura léxica de la consulta, cantidad de fragmentos, puntuación máxima normalizada, diversidad de fuentes y answerabilidad. Si esa confianza supera el umbral $\theta_c = 0,65$, el pipeline pasa directamente a la generación.

Cuando la confianza no alcanza el umbral, el pipeline ejecuta una recuperación sobre el caché web antes de invocar la búsqueda externa. Si los fragmentos del caché elevan la confianza por encima del umbral, la búsqueda externa no se ejecuta. Esta secuencia de evaluaciones en cascada (corpus $\rightarrow$ caché $\rightarrow$ web externa) minimiza el número de llamadas al motor de búsqueda externo: solo se recurre a él cuando ni el corpus ni las búsquedas anteriores contienen la información necesaria.

Si la búsqueda externa se ejecuta, el orquestador descarga, chunkea e indexa los documentos encontrados en el caché web, y el pipeline realiza una segunda recuperación sobre ese caché para obtener los fragmentos recién indexados. En todas las variantes del flujo, el conjunto final de fragmentos es la unión deduplicada de lo que esté disponible en cada momento.

4 Selección de fragmentos para el prompt

El método `_select_prompt_chunks` decide qué fragmentos se incluyen en el prompt enviado al modelo. Si el reordenador está activo, puntúa cada par (consulta, fragmento) con el cross-encoder y selecciona los `context_chunks=15` mejor posicionados. Si el reordenador no está activo, toma los primeros 15 fragmentos según el orden del recuperador híbrido.

La deduplicación entre fragmentos del corpus y del caché web se realiza antes de la selección: si el mismo `chunk_id` aparece en ambas fuentes, se conserva una única copia. Esto evita que el prompt repita información y que el modelo generativo reciba señales contradictorias sobre la misma fuente.

5 Construcción del prompt

Los fragmentos seleccionados se formatean en un contexto estructurado que se incluye en el mensaje del usuario. Cada fragmento aparece numerado con su título y URL de origen:

```
[1] Título del documento
Source: https://ejemplo.com/doc
```

```
Texto del fragmento...
```

```
---
```

```
[2] Otro documento
...
```

El mensaje del sistema define el comportamiento esperado del modelo en español, con cuatro instrucciones operativas concretas: responder siempre en español independientemente del idioma del contexto; utilizar exclusivamente la información de los fragmentos proporcionados; indicar qué aspectos no están cubiertos si la información es parcial; y responder que no hay información suficiente solo cuando los fragmentos no tienen ninguna relación con la pregunta. Esta última condición es más exigente que un simple “no sé”: el sistema distingue entre la ausencia total de información relevante y la presencia de información parcial, en cuyo caso el modelo debe usar lo disponible y señalar las lagunas.

La instrucción de usar bloques markdown para código garantiza que los fragmentos de código presentes en los documentos técnicos se preserven formateados en la respuesta.

6 Cliente LLM

El `LLMClient` implementa la interfaz de *chat completions* compatible con OpenAI, lo que permite conectar el sistema a cualquier proveedor que exponga ese protocolo: Groq, OpenAI, Anthropic vía proxy, modelos locales con LM Studio u Ollama, entre otros. El proveedor por defecto es Groq con el modelo `llama-3.3-70b-versatile`, configurado mediante la variable `LLM_BASE_URL`.

La llamada al modelo envía un payload con cuatro parámetros: el nombre del modelo, los mensajes (sistema y usuario), el máximo de tokens a generar (`LLM_MAX_TOKENS=1024`) y la temperatura (`LLM_TEMPERATURE=0.1`). La temperatura baja reduce la variabilidad de las respuestas, lo que es apropiado para un sistema de documentación técnica donde la consistencia es más valiosa que la creatividad. El timeout por defecto es de 60 segundos.

La respuesta del modelo se parsea extrayendo el contenido del primer choice y los conteos de tokens de uso (`prompt_tokens` y `completion_tokens`). Estos conteos se incluyen en el resultado devuelto al cliente, lo que permite monitorizar el consumo de tokens por consulta sin instrumentación adicional.

El cliente expone dos métodos de actualización en caliente (`update_settings` y `update_connection`) que permiten cambiar el modelo, la temperatura, el número máximo de tokens, la URL base o la

clave API sin reiniciar el servidor. Estos métodos son invocados por la API de configuración cuando el operador ajusta parámetros en tiempo de ejecución.

7 Trazabilidad de fuentes

Cada fragmento incluido en el prompt genera un objeto **RAGSource** en la respuesta final. Este objeto contiene el identificador del fragmento, la URL del documento, el título, el tipo de fuente y el método de recuperación:

- **source_type**: distingue entre **"corpus"** (fragmentos del índice curado) y **"web_cache"** (fragmentos obtenidos de búsquedas web). Un fragmento se clasifica como **web_cache** si su **source_id** comienza con **"web:"** o si su **breadcrumb** es **"web-search"**.
- **retrieval_method**: indica si el fragmento fue encontrado por BM25 (**"bm25"**), por el recuperador vectorial (**"vector"**), por ambos (**"hybrid"**), o proviene del caché web (**"web_cache"**).

Esta información permite al usuario evaluar la procedencia de cada parte de la respuesta. Un fragmento recuperado por ambos señales (BM25 y vectorial) ofrece mayor confianza que uno encontrado solo por uno de los dos. Un fragmento marcado como **web_cache** indica que el corpus local no contenía esa información y fue necesario recurrir a la web.

8 Historial de conversación

El pipeline soporta historial de conversación opcional mediante un identificador de sesión (**session_id**). Cuando el cliente incluye un **session_id** válido en la solicitud (cadena alfanumérica de 8 a 128 caracteres), el pipeline almacena el par (pregunta, respuesta) en Redis bajo la clave **rag:chat:history:{session_id}**.

El historial se almacena como una lista de mensajes tipados: los mensajes del usuario tienen **type="user"** y los del asistente **type="assistant"** con sus fuentes y el modelo utilizado. Cada entrada incluye un identificador único y una marca temporal en formato ISO. El TTL por defecto es de 7 días (**CHAT_HISTORY_TTL_SECONDS=604800**), renovado en cada intercambio.

El historial es consultable mediante el endpoint **GET /api/v1/rag/history/{session_id}**, que devuelve la secuencia completa de mensajes de la sesión. El módulo RAG no inyecta el historial en el prompt del modelo en la implementación actual: cada consulta se responde de forma independiente. El historial existe como registro de la interacción para el cliente, no como contexto adicional para el modelo.

9 Configuración en tiempo de ejecución

Una de las características distintivas del módulo RAG es que todos sus parámetros operativos son ajustables en tiempo de ejecución sin reiniciar el servidor. La API de configuración (**POST /api/v1/config**) acepta un objeto con los parámetros a modificar y aplica los cambios de forma inmediata sobre las instancias activas del pipeline.

Los parámetros modificables incluyen los pesos de fusión híbrida (BM25 y vectorial), el modelo y temperatura del LLM, la URL base y clave API del proveedor, el número máximo de tokens, el número de fragmentos de contexto, el número de candidatos para el reordenador, los 15 umbrales y pesos del detector de insuficiencia, y la habilitación o deshabilitación de HyDE y del reordenador. Los cambios de pesos del detector validan que la suma de los cinco pesos sea exactamente 1,0 antes de aplicarlos.

Los cambios se persisten en un archivo `pipeline_config.json`, de modo que sobreviven a un reinicio del servidor. La configuración leída al arrancar del archivo sobrescribe los valores por defecto del entorno, lo que permite mantener un estado de configuración estable entre reinicios sin modificar las variables de entorno.

Variable	Default	Descripción
LLM_BASE_URL	<code>https://api.groq.com/openai/v1</code>	Proveedor LLM
LLM_MODEL	<code>llama-3.3-70b-versatile</code>	Modelo de generación
LLM_MAX_TOKENS	<code>1024</code>	Tokens máximos en la respuesta
LLM_TEMPERATURE	<code>0.1</code>	Temperatura de muestreo
LLM_CONTEXT_CHUNKS	<code>15</code>	Fragmentos en el prompt
LLM_TIMEOUT	<code>60.0</code>	Timeout de llamada al LLM (s)
RERANKER_ENABLED	<code>true</code>	Activa el reordenador
RERANKER_CANDIDATE_K	<code>30</code>	Candidatos para reordenamiento
HYDE_ENABLED	<code>false</code>	Expansión hipotética de consulta
REDIS_URL	<code>redis://redis:6379/0</code>	Almacén del historial
CHAT_HISTORY_TTL_SECONDS	<code>604800</code>	TTL del historial (7 días)

10 Resultado devuelto al cliente

El endpoint `POST /api/v1/rag/query` devuelve un objeto con todos los campos necesarios para que el cliente reconstruya la procedencia completa de la respuesta:

- **answer**: texto de la respuesta generada por el LLM.
- **sources**: lista de `RAGSource` con `chunk_id`, `url`, `title`, `source_type` y `retrieval_method`.
- **model**: nombre exacto del modelo que generó la respuesta.
- **prompt_tokens** y **completion_tokens**: conteos de tokens de la llamada.
- **cache_searched**: indica si se buscó en el caché web.
- **cache_hits**: número de fragmentos encontrados en el caché web.
- **external_search_executed**: indica si se ejecutó una búsqueda web externa.
- **external_indexed_count**: número de fragmentos indexados de la búsqueda web.
- **web_search**: objeto con los hits y documentos de la búsqueda externa, si aplica.

Esta trazabilidad completa permite auditar cualquier respuesta del sistema: es posible identificar exactamente qué fragmentos influyeron en la respuesta, de dónde provienen y cómo fueron recuperados, sin requerir instrumentación adicional ni acceso a los logs del servidor.

11 Justificación técnica de la solución

La arquitectura del módulo RAG responde a tres requisitos concretos del dominio: responder sobre documentación técnica especializada que no está en el conocimiento de entrenamiento de los modelos disponibles, hacerlo en español independientemente del idioma del corpus, y mantener la respuesta fundamentada en fuentes verificables.

Un modelo de lenguaje sin contexto no puede cumplir el primer requisito por definición. Un sistema de recuperación sin generación produciría fragmentos de texto en lugar de respuestas coherentes. La combinación RAG resuelve ambos problemas: la recuperación garantiza que la evidencia correcta esté disponible, y el modelo transforma esa evidencia en una respuesta articulada en el idioma del usuario.

La decisión de implementar la evaluación de suficiencia dentro del pipeline RAG, en lugar de hacerla una etapa externa previa, responde a que la calidad de la recuperación solo puede evaluarse en el contexto de una consulta concreta. No existe un umbral de calidad del corpus válido en general: un corpus con diez fragmentos puede ser suficiente para una pregunta y completamente insuficiente para otra. Al evaluar después de cada recuperación, el sistema adapta su comportamiento a cada consulta en lugar de aplicar una política fija.

La configuración en caliente de todos los parámetros operativos responde a la naturaleza exploratoria del ajuste de sistemas RAG. Los valores óptimos de temperatura, número de fragmentos, pesos de fusión o umbrales de insuficiencia dependen del corpus específico y del perfil de consultas, y no son conocidos a priori. Permitir el ajuste sin reiniciar el servidor reduce el ciclo de experimentación y permite ajustar el comportamiento del sistema en producción en respuesta a consultas reales sin interrumpir el servicio.

12 Limitaciones del módulo

El módulo no incorpora el historial de conversación en el prompt del modelo. Cada consulta se responde de forma independiente, lo que significa que el sistema no puede responder preguntas de seguimiento que dependan de respuestas anteriores de la misma sesión. El historial existe como registro de la interacción pero no influye en la generación.

El prompt del sistema instruye al modelo a responder solo con la información de los fragmentos proporcionados, pero no existe ningún mecanismo de verificación automática de que la respuesta generada se limite a esa información. Un modelo que no siga las instrucciones del sistema podría generar afirmaciones que no aparecen en los fragmentos sin que el sistema lo detecte.

El conteo de tokens reportado en la respuesta proviene del campo `usage` de la respuesta del proveedor LLM. Si el proveedor no incluye ese campo o lo incluye incompleto, los valores `prompt_tokens` y `completion_tokens` se registran como cero sin generar error, lo que puede producir métricas de consumo incorrectas de forma silenciosa.

La compatibilidad con la interfaz OpenAI permite conectar proveedores alternativos, pero asume que todos implementan correctamente el mismo protocolo de *chat completions*. Un proveedor con respuestas malformadas o con campos faltantes puede producir errores en tiempo de ejecución que no están cubiertos por el manejo de excepciones actual.